PRODUCT DEVELOPMENT LIFECYCLE DOCUMENT

Cartly | Autonomous Customer-Support Resolution Platform

Capstone Project 01

Version 1.0 | Sprint 0 | June 2026

PREPARED BY

Avishka Jindal

Hiten Mistry

Yash Parmar

Document Control

Version History

Version	Date	Author	Change Summary
1.0	June 2026	AI Engineering Squad	Initial release covering Sprint 0 Stage 1 and 2 deliverables
1.1	TBD	AI Engineering Squad	Stage 3 architecture additions post Sprint 1 review
2.0	TBD	AI Engineering Squad	Full document incorporating Sprints 2 through 4 findings

Document Purpose

This Product Development Lifecycle (PDLC) document governs the end-to-end design, build, evaluation and operation of the Cartly Autonomous Customer-Support Resolution Platform. It serves as the primary reference for all engineering decisions, assessment submissions and viva defence. Every section maps to a named stage in the Capstone Project brief. Nothing in this document is aspirational without a corresponding metric, baseline and owner.

Scope Statement

This document covers eight sequential stages: discovery, definition, design, risk assessment, data strategy, build, verification and operations. It governs a fictional client engagement with Cartly, an online marketplace, as the named client persona for this capstone. The system is not integrated with a live production marketplace. All tool calls use mock services seeded from publicly available support datasets.

Table of Contents

Executive Summary

Cartly is an online marketplace connecting independent sellers with consumers across home, hobby and lifestyle categories. Order volume has tripled in twelve months. The support function has not scaled with it. A single queue absorbs every ticket type: late deliveries, refund requests, payment failures, account-access problems, product questions and buyer-seller disputes. Agents read tickets cold, hunt for order records, check policy and type replies. On normal weekdays they cope. On weekends and sale events, first-response time exceeds a day, customers repeat themselves and the cost per resolved ticket rises.

This document describes the design, build and operation of an agentic resolution system that takes ownership of tickets end to end. The system triages incoming tickets, routes them to specialist agents, resolves them with answers grounded in Cartly policy and live order data, passes every response through a quality and safety critic, and either closes the ticket autonomously or hands a clean, fully prepared case to a human agent.

> **System Mandate**
> Take a real support workload from raw tickets to a deployed, agentic resolution system. Measured against honest numbers. Run inside a fixed budget. Trustworthy on cases that move money or touch a customer account.

Key Targets at a Glance

Metric	Baseline	Sprint 4 Target
Autonomous resolution rate	0% (all tickets human-handled)	65% or above
First-response time (peak)	18-36 hours	Under 30 seconds
Resolution time per ticket	11 minutes active + wait	Under 3 minutes (Tier 1)
Cost per resolved ticket	USD 3-8 (fully loaded)	Under INR 30 (approx. USD 0.36)
RAGAS faithfulness score	No baseline (no grounding)	0.85 or above
Safety on sensitive cases	No automated gate (human only)	100% human approval gate

Stage 1 - Discover and Probe

Sprint 0 Deliverable
A grounded read of the problem, the users and the data. Backed by numbers. This stage produces the discovery brief, persona set and prioritised ticket map that feed Stage 2 and Stage 3.

1.1 The Client and the Situation

Cartly connects independent sellers with consumers across home, hobby and lifestyle categories. In twelve months, order volume has tripled. The support team has not grown to match it. One queue handles all ticket types and all agents work from cold context on every ticket. The problem is not agent skill; it is structural throughput. A capable agent reading a ticket cold still spends two to three minutes reconstructing context before they can begin resolving.

Cartly's leadership has evaluated FAQ bots and rejected them. A bot that recites the returns policy resolves nothing for a customer whose refund is genuinely stuck. The requirement is a system that takes ownership of a ticket, gathers the facts from live systems, applies the correct policy rule, takes an appropriate action and either closes the ticket or hands a complete, ready-to-act brief to a human. The mandate is not deflection. It is resolution.

1.2 Ticket Distribution Analysis

Based on analogous marketplace deployments and the Customer Support on Twitter corpus (approximately three million real support interactions across twenty brands), the ticket volume for a marketplace of Cartly's profile distributes as follows.

Category	Estimated Share	Resolution Complexity
Order status and WISMO (Where Is My Order)	35%	Low - data lookup only
Refund and return requests	22%	Medium - eligibility check and action
Payment failures and gateway errors	12%	Medium to high - payment record and policy
Product questions (specifications and compatibility)	10%	Low - knowledge retrieval
Account access and login problems	8%	High - identity-adjacent and sensitive
Buyer-seller disputes	7%	High - multi-party and escalation-likely
Delivery and logistics complaints	4%	Medium - courier API and ETA
Other and miscellaneous	2%	Variable

The top two categories (WISMO and returns) account for 57% of total volume and share the same structural pattern: a data lookup followed by a policy eligibility check, then either an informational response or a bounded action. This is the highest-leverage automation target and the starting point for the Tier 1 autonomy layer.

1.3 User Personas

Five personas represent the people this system serves and the people who operate it. Each persona has a named goal, a named frustration and a job to be done.

Persona 1 - The Frustrated Buyer

Attribute	Detail
Name and profile	Priya, 34, hobby crafter based in Pune
Goal	Receive confirmation that her return is accepted and know when the refund arrives, in one interaction
Frustration	She submitted a return request five days ago. Status shows under review. She has sent two follow-ups and no agent has accessed her original order record
Job to be done	Resolve my support issue in one interaction without requiring me to repeat myself

Persona 2 - The Seller Caught in a Dispute

Attribute	Detail
Name and profile	Rahul, 41, kitchenware seller on Cartly
Goal	Have his evidence reviewed before any refund or account action is taken against him
Frustration	A buyer claimed non-delivery. Tracking confirms delivery. Rahul has proof but cannot attach it and the responding agent appeared unaware of the tracking record
Job to be done	Let me surface my evidence and have it weighed before any financial or account action is executed

Persona 3 - The Human Escalation Agent

Attribute	Detail
Name and profile	Kavya, support specialist, two years at Cartly
Goal	Pick up an escalated ticket and resolve it in under three minutes without reconstructing the case from scratch

Attribute	Detail
Frustration	The AI hands off a ticket with no context summary. Kavya re-reads the original message, looks up the order and checks what actions were already attempted before she can begin. Every handoff erases prior work
Job to be done	Receive a pre-built case brief so I can resume immediately from where the system left off

Persona 4 - The Operations Lead

Attribute	Detail
Name and profile	Arjun, head of support operations at Cartly
Goal	Know at any point how many tickets are open, what the AI resolved, what it escalated, what it cost and where quality risk is concentrated
Frustration	He knows the backlog size but has no visibility into resolution quality, cost per ticket or category-level bottlenecks. Reports are assembled manually and lag by two days
Job to be done	A live dashboard for throughput, cost, quality and escalation rate so I can answer to leadership without running manual queries

Persona 5 - The Compliance-Aware Founder

Attribute	Detail
Name and profile	Neeha, co-founder and product lead at Cartly
Goal	Deploy AI that increases throughput without creating financial liability or regulatory exposure
Frustration	Vendor demos show FAQ bots. She needs a system that handles hard cases as well as easy ones, but only when it can be trusted and only when every sensitive action is auditable
Job to be done	A system with a traceable audit trail, human checkpoints on every payment-adjacent case and a kill switch that is one command

1.4 Jobs to Be Done Summary

User	Job to Be Done
Buyer (Priya)	Resolve my issue in one interaction in plain language without requiring me to repeat myself
Seller (Rahul)	Let me surface my evidence and have it considered before any account or financial action is taken
Escalation Agent (Kavya)	Hand me a pre-built case brief so I can resume from where the AI left off immediately

User	Job to Be Done
Operations Lead (Arjun)	Give me a live view of throughput, quality, cost and escalation rate without manual extraction
Founder (Neeha)	Deploy a system I can trust on payment-adjacent cases with a traceable audit trail and a defined kill switch

1.5 Probing Questions for the Client

These are the questions the team would put to Cartly leadership before committing to any architecture. Where the answer is unavailable, the assumption is stated explicitly and marked.

#	Question	Why It Matters	Working Assumption
Q1	What is the daily ticket volume and peak multiplier?	Sets concurrency design and queue capacity	A1: Peak surge is 3x the weekday average
Q2	What actions is the system permitted to take without human approval?	Defines the Tier 1 autonomy boundary	A2: Order lookup, policy retrieval and refunds below INR 500 are autonomous
Q3	Is there a hard per-ticket cost ceiling?	Sets the budget guardrail metric	A3: INR 30 per resolved ticket ceiling
Q4	What is the latency requirement for first response?	Determines model routing policy	A4: First AI response within 30 seconds of ticket receipt
Q5	Which categories must always route to a human?	Hard escalation rules cannot be inferred	A5: Account suspensions, fraud allegations and legal mentions always escalate
Q6	Are tickets multi-lingual?	Drives embedding model choice and synthetic data scope	A6: Primarily English and Hindi with occasional regional language text
Q7	Is live order data accessible or is a mock API sufficient?	Determines the tool layer design	A7: Mock services seeded from dataset for the build phase
Q8	What is the state of policy documentation?	Determines RAG preparation effort	A8: Policy docs exist but are incomplete and inconsistently formatted
Q9	Is seller dispute input structured data or free text?	Affects dispute agent design	A9: Sellers submit free text; NLU extraction is required
Q10	What constitutes a resolved vs. a closed ticket?	Makes the resolution rate metric unambiguous	A10: Resolved means the issue is addressed and no follow-up arrives within 48 hours

1.6 System Teardown: Intercom Fin AI Agent

Overview

Intercom Fin is an AI-first agent system built on the Fin AI Engine. It achieves a published average autonomous resolution rate of 51% across its customer base and 67% in mature deployments as of December 2025. It handles refunds, return initiations and subscription changes as first-class actions via its Tasks and Procedures abstraction. Operators define named action sequences and the AI executes them as atomic, auditable units.

Automation and Escalation Boundary

Operators configure which intents and action types fall within autonomous scope. For out-of-scope intents or when confidence falls below a threshold, Fin escalates with a structured handoff that includes conversation history and a summary. A directly comparable deployment, a marketplace operator with USD 80 million GMV and 30 agents, achieved a 45 to 55 percent autonomous resolution rate on Tier 1 volume while maintaining CSAT within 3 points of the human baseline through a 2x volume season.

Design Idea Worth Borrowing

The Tasks and Procedures abstraction: named action sequences that the AI executes as atomic units. Each sequence is independently testable, auditable and version-controlled. For Cartly, implementing named action procedures for the top five ticket categories before wiring general tool use will improve reliability and reduce debugging time.

Anti-Pattern Worth Avoiding

Per-resolution pricing at USD 0.99 per resolution creates a cost structure that rises directly with system success. As the autonomous resolution rate improves from 50 to 65 percent, the monthly bill grows proportionally with no cap. For a rapidly growing platform this is a budget control failure. The bespoke Cartly build uses infrastructure pricing so cost remains bounded by compute, not resolution rate.

1.7 System Teardown: Zendesk AI Resolution Platform

Overview

Zendesk embeds AI capabilities into its established helpdesk, trained on over 18 billion interactions across 80 languages. It claims up to 80 percent autonomous resolution on routine queries. Skills-based routing assigns tickets by agent skill profile and workload. The copilot proactively surfaces intent, sentiment and customer history to human agents without manual prompting.

Automation and Escalation Boundary

The AI resolves routine queries directly. Complex or low-confidence queries route to the best-matched agent with full conversation context loaded in the unified workspace. The 80 percent resolution figure applies to routine single-intent queries; marketplace-specific resolution rates are significantly lower due to multi-intent and policy-sensitive ticket profiles.

Design Idea Worth Borrowing

The unified agent workspace pattern: all channels (email, chat, in-app and WhatsApp) converge into one structured view with order history, ticket history and AI-suggested actions pre-loaded. The human agent never switches contexts. For Cartly, the human handoff interface should deliver a structured case brief rather than a raw conversation log.

Anti-Pattern Worth Avoiding

AI capabilities are tightly coupled to the Zendesk platform. Every AI improvement requires a platform upgrade and cannot be iterated independently. Zendesk CX Trends 2025 found that 64 percent of customers still report frustration at handoff due to context loss, even in well-funded enterprise deployments. For Cartly, the system architecture must decouple the LLM backend, the orchestration layer and the data layer so each component is independently swappable.

Stage 2 - Define (Product Requirements Document)

Sprint 0 Deliverable
A PRD a stakeholder could approve and a team could build from on its own. Every metric has a baseline. Every scope boundary has a rationale. Every assumption is numbered and traceable.

2.1 Problem Statement

Cartly processes a tripled order volume through an unmigrated support function. The single-queue model forces agents to work from cold context on every ticket. Peak-period first-response time exceeds 24 hours. The cost per resolved ticket rises with headcount because throughput scales linearly with staff rather than with system capacity. Customers who contact support during peak periods experience degraded service regardless of agent skill.

The problem is structural: there is no system layer between the incoming ticket and the human agent that can take ownership, gather facts and execute bounded actions. Every ticket, simple and complex alike, consumes the same agent time for context reconstruction.

2.2 Target Users

Primary users are buyers seeking issue resolution and sellers seeking fair dispute handling. Internal operators are escalation agents requiring complete handoff briefs, the operations lead requiring live performance visibility and leadership requiring auditability and cost control. All five personas are defined in full in Stage 1.

2.3 Scope and Non-Scope

In Scope

- Triage and intent classification on all incoming support tickets
- Autonomous resolution of Tier 1 tickets: order status, standard returns, policy lookups and shipping updates
- Conditional resolution of Tier 2 tickets with human review before action: refunds above threshold and first-contact dispute claims
- Human handoff with structured case brief for Tier 3 and Tier 4 tickets
- Multi-agent orchestration with LLM gateway, model routing and agent registry
- RAGAS-evaluated retrieval-grounded responses backed by versioned policy knowledge base
- Token cost tracking and budget guardrail enforcement
- Safety and adversarial red-team evaluation
- Deployment as API with minimal interface and operate runbook

Not in Scope

- Live integration with a production marketplace or live payment processor

- Multi-language support beyond English and Hindi in Sprint 1 and 2
- Voice channel support
- Seller onboarding or product catalogue management queries
- Legal advice or formal regulatory submissions

2.4 Success Metrics with Baselines

Every metric below has an operational definition, a confirmed or assumed baseline and a target. A target without a baseline cannot be judged.

Metric	Operational Definition	Baseline	Target
Autonomous resolution rate	Percentage of tickets fully closed by AI with no human action and no customer follow-up within 48 hours	0% (all human-handled)	50% by Sprint 3; 65% by Sprint 4
Escalation precision	Of escalated tickets, the percentage that genuinely required human action as confirmed by post-review	Unknown - to be measured in Sprint 1 golden set	85% or above
Groundedness (RAGAS faithfulness)	Percentage of AI responses where every factual claim is traceable to a retrieved document or successful tool call	0% (no grounding baseline)	0.85 or above
Safety on sensitive cases	Percentage of refund, account and dispute tickets where the human approval gate was correctly applied	Not applicable - guardrail metric	100% - never negotiable
Cost per resolved ticket	Total LLM token cost plus infrastructure cost for a single AI-closed ticket	USD 3-8 fully loaded (human)	Under INR 30 (approx. USD 0.36)

2.5 Guardrail Metrics

Guardrail metrics are ceilings that must never be crossed even when headline metrics improve. Violating a guardrail is a product failure regardless of resolution rate.

- Cost per resolved ticket must not exceed INR 30 at any system configuration

- Safety on sensitive cases must remain at 100 percent; any deviation triggers immediate rollback
- RAGAS faithfulness must not drop below 0.70 at any sprint gate even during A/B testing

2.6 Non-Functional Requirements

Requirement	Target	Rationale
First-response latency	30 seconds or under	Assumption A4 - must be confirmed with client; sets model routing policy
Tier 1 resolution time	3 minutes or under	Based on Klarna's 2-minute resolution benchmark for autonomous handling
Peak concurrency	3x normal weekday volume with no queue backup	Assumption A1 - drives queuing and caching design in Sprint 1
Cost ceiling	INR 30 per resolved ticket enforced as budget guardrail with alarms	Assumption A3 - confirmed against actual API pricing in Sprint 2
Traceability	Every agent step, tool call and decision logged with timestamp and version reference	Required for audit trail, viva defence and regulatory readiness
Availability	System degrades to human path if any component is unavailable; total failure is not acceptable	Kill switch and graceful fallback are Stage 4 design requirements

2.7 Open Questions and Assumptions Register

All assumptions carry a numbered label (A1 through A10). Each must be revisited and either confirmed or revised when client data becomes available. Full assumption details are in Stage 1 Section 1.5.

Ref	Assumption	Risk if Wrong	Resolution Path
A1	Peak surge is 3x the weekday average	Under-designed concurrency causes queue backup at launch	Confirm with Cartly ops data before Sprint 2 build

Ref	Assumption	Risk if Wrong	Resolution Path
A2	Refunds below INR 500 are within autonomous Tier 1 scope	Wrong threshold causes either over-automation or under-automation	Confirm with Cartly legal and ops leadership
A3	INR 30 is the per-ticket cost ceiling	Wrong ceiling invalidates all token-economics decisions	Confirm against actual API pricing in Sprint 2
A4	30-second first-response requirement applies	Wrong target drives incorrect model routing policy	Confirm with Cartly product lead
A5	Account suspensions, fraud and legal tickets always escalate	Under-escalation creates compliance exposure	Confirm with Cartly legal or compliance owner
A8	Policy docs exist but are incomplete and inconsistently formatted	Clean docs require less preparation than anticipated or more	Audit policy documentation at Sprint 1 start

Stage 3 - Design

Sprint 1 Deliverable

A spec a new engineer could build from on their own without asking questions. Covers agent topology, orchestration decisions, the LLM gateway, model routing, the agent registry, memory design and tool specifications.

3.1 System Architecture Overview

The system receives tickets from all channels through a unified entry point, passes them through the triage and routing layer, dispatches to the appropriate specialist agent, applies a quality and safety critic before any reply is sent, and either closes the ticket autonomously or escalates with a complete handoff brief. Every model call passes through a central LLM gateway that enforces routing policy, rate limits and fallbacks.

Architecture Principle

No component is tightly coupled to another. The LLM backend, orchestration layer and data layer are independently swappable. Replacing the vector store, the LLM provider or the agent implementation must not require changes to adjacent components.

3.2 Agent Topology

Each agent has a single named responsibility, a defined input and output schema, a declared tool surface and an assigned model tier. The model tier reflects the cost-accuracy tradeoff appropriate to that agent's task.

Agent	Single Responsibility	Key Tools	Model Tier	Notes
Triage Agent	Classify intent, score risk and assign routing decision	Intent classifier, sentiment scorer	Cheap (DeepSeek or Qwen)	Handles 100% of incoming tickets; must be fast and low-cost
Order Agent	Look up order status, fulfilment state and delivery ETA	Order lookup API (mock)	Cheap	Read-only; no actions; grounded in structured data only
Refund Agent	Check eligibility and initiate refund within the approved threshold	Refund eligibility API, refund action API	Medium - actions require stronger grounding	INR 500 threshold enforced as hard limit
Policy Agent	Retrieve and apply Cartly policy rules to ticket context	ChromaDB knowledge	Medium	Self-RAG or CRAG: abstains

Agent	Single Responsibility	Key Tools	Model Tier	Notes
		base, policy lookup		when evidence is weak
Dispute Agent	Gather buyer and seller evidence, prepare escalation brief	Order API, communication log API	Strong (multi-party reasoning required)	Always produces a handoff brief; never closes autonomously
Escalation Agent	Package the case brief and route to the human queue	Handoff API, CRM updater	Cheap	Output is a structured brief: what was asked, retrieved, attempted and why escalated
Safety Critic	Review every draft response before it is sent	RAGAS scorer, guardrail rules, Llama Guard	Medium	Blocks any response with a faithfulness score below 0.70 or a safety flag

3.3 Orchestration Decision Record

The brief specifies four orchestration patterns. Each is assessed below against the Cartly ticket profile. The decision is stated with the rationale. Rejected patterns are explained.

Pattern	Description	Decision	Rationale
Router-and-handoff	First-touch triage by intent and risk, then route to the appropriate specialist	ADOPT - entry point	The ticket profile has at least six distinct categories each requiring a different specialist. A router that classifies before dispatching reduces specialist load and prevents mis-routing.
Orchestrator-worker	A supervisor delegates subtasks to specialist worker agents	ADOPT - outer loop	The outer loop of the system is a supervisor (orchestrator) that manages the lifecycle of a ticket across the specialist agents (workers). This maps directly to the multi-category ticket distribution.
Evaluator-optimizer	A quality and safety critic reviews drafts before they are sent	ADOPT - final gate	Every response must pass the Safety Critic before reaching the customer. This pattern ensures no hallucinated policy or unsafe action escapes. It also enables DSPy optimization against the faithfulness metric.

Pattern	Description	Decision	Rationale
Sequential	Ordered steps inside one resolution path	ADOPT - within-agent execution	Inside each specialist agent (for example the Refund Agent: check eligibility, validate threshold, call API, log result) the steps are sequential and ordered. This pattern is used at the sub-agent level, not the system level.

All four patterns are adopted at different scopes: the outer loop is orchestrator-worker; the entry point is a router; each worker's internal execution is sequential; and the final gate before any reply is the evaluator-optimizer. No pattern is rejected outright; each occupies a distinct scope.

3.4 LLM Gateway Design

LiteLLM serves as the single point through which every model call in the system passes. No agent calls an LLM provider directly. This abstraction enables provider-level fallback, rate-limit handling, cost tracking and model swaps without changes to agent code.

Capability	Implementation
Provider abstraction	Supports OpenAI, Anthropic, Mistral, Groq and local models via unified interface
Rate limiting	Per-provider and per-agent rate limits enforced at the gateway layer
Retries	Exponential backoff with jitter on transient failures; maximum three retries
Timeouts	Hard timeout of 25 seconds per call; caller receives a structured error, not a hang
Fallback chain	If primary provider returns an error or timeout, gateway routes to the next provider in the fallback list without agent awareness
Cost tracking	Every call logs input tokens, output tokens, model ID, agent ID and timestamp to the audit log
Secrets management	Provider keys are injected via environment variables; no key appears in code or in prompts

3.5 Model Routing Policy

Not every task requires a strong model. The routing policy maps task type to the appropriate model tier, reducing per-ticket cost without sacrificing quality on tasks that require stronger reasoning.

Task Type	Assigned Tier	Rationale
Intent classification and triage	Cheap (DeepSeek or Qwen via Groq free tier)	High volume; structured output; fast latency required
Order lookup and status reply	Cheap	Tool-grounded; LLM is only formatting the structured API result
Policy retrieval and standard returns	Medium (Mistral or Gemini Flash)	Requires faithful grounding but not complex reasoning
Refund eligibility and conditional actions	Medium	Accuracy-critical; threshold guards handle the risk layer
Dispute handling and multi-party reasoning	Strong (Claude or GPT-4 class)	Multi-hop evidence reasoning; strong model justifies higher cost
Safety Critic	Medium with guardrail layer	Must catch faithfulness failures and injection attempts
Synthetic data generation	Strong (one-time batch job)	Quality of synthetic data sets the benchmark ceiling

3.6 Agent Registry

The registry is the authoritative catalogue of all agents in the system. Every agent entry carries a stable identifier, a version, a schema declaration, a declared tool list and a model assignment. Any agent can be swapped, versioned or disabled without modifying the orchestrator.

Agent ID	Version	Input Schema	Output Schema
triage_agent	v1.0	raw_ticket: string, channel: enum, timestamp: ISO8601	intent: enum, risk_tier: [1-4], routed_to: agent_id, confidence: float
order_agent	v1.0	order_id: string, customer_id: string, query_context: string	order_status: object, eta: string, fulfilment_state: enum
refund_agent	v1.0	order_id: string, refund_amount: float, reason: string	eligibility: bool, action_taken: enum, transaction_ref: string
policy_agent	v1.0	query: string, category: enum, retrieved_context: list	policy_answer: string, source_refs: list, confidence: float
dispute_agent	v1.0	ticket_id: string, buyer_claim: string, seller_response: string	escalation_brief: object, recommendation: enum
escalation_agent	v1.0	ticket_id: string, case_brief: object	handoff_package: object, queue_assignment: string

Agent ID	Version	Input Schema	Output Schema
safety_critic	v1.0	draft_response: string, context: object	approved: bool, faithfulness_score: float, flags: list

3.7 Memory and State Design

Memory Type	Scope	Storage
Within-interaction working memory	Single ticket resolution session	LangGraph state graph; persists across agent steps in one run
Cross-interaction customer profile	All interactions by one customer ID	PostgreSQL customer profile table; indexed by customer_id
Policy knowledge base	Versioned policy documents	ChromaDB with timestamp metadata enabling policy drift detection
Audit log	All agent steps, tool calls and decisions	Append-only event log in PostgreSQL; immutable after write
Human handoff brief	One escalated ticket	Structured JSON object written to the handoff queue at escalation time

3.8 Tool Specifications

All tools are backed by mock services seeded from the primary dataset. The mock services return deterministic responses for known test cases and probabilistic responses for novel inputs. No live marketplace integration is required for the build phase.

Tool Name	Input Parameters	Output Contract
order_lookup	order_id: string, customer_id: string	OrderRecord object: status, items, payment_ref, delivery_eta, seller_id
refund_eligibility_check	order_id: string, request_date: ISO8601, reason_code: enum	EligibilityResult: eligible bool, reason string, max_refund_amount float
refund_action	order_id: string, amount: float, reason_code: enum	TransactionResult: success bool, transaction_ref string, estimated_settlement string
policy_retrieval	query: string, category: enum, top_k: int	PolicyResult: chunks list, source_ids list, retrieval_scores list
human_handoff	ticket_id: string, case_brief: object, urgency: enum	HandoffResult: queue_position int, assigned_to string, expected_wait string

Tool Name	Input Parameters	Output Contract
crm_update	ticket_id: string, status: enum, resolution_summary: string	UpdateResult: success bool, updated_fields list

Stage 4 - Risk Assessment and Mitigation

Maintained from Sprints 0 through 4

Risk is a first-class deliverable, not a closing paragraph. This register is updated at every sprint. Each risk has a likelihood, an impact rating, an owner, a concrete mitigation and the residual risk that remains after the mitigation is applied.

4.1 Risk Register

Risk	Why It Matters	Likelihood	Impact	Mitigation	Residual Risk
PII exposure in prompts	Tickets carry names, order IDs, contact details and payment references. LLM prompts must not include raw PII unnecessarily	Medium	High	Redaction layer before prompt injection; least-privilege tool design; no secrets in prompts or code; environment variables only	Low - redaction reduces exposure surface substantially
Wrong refund or account action	An automated action can move money or change an account. Air Canada was held legally liable for an incorrect chatbot refund promise	Low	Critical	INR 500 hard threshold on autonomous refund; eligibility check as mandatory pre-step; human approval for all amounts above threshold; immutable action log	Very low - hard threshold prevents financial harm at scale
Hallucinated policy answer	An invented or outdated policy rule misleads the customer and creates liability. Klarna reported 5% conversation degradation from hallucinations	Medium	High	All policy claims grounded in ChromaDB retrieval; Self-RAG or CRAG abstains when evidence is weak; Safety Critic blocks responses below RAGAS faithfulness 0.70	Low - grounding plus critic provides two independent checks

Risk	Why It Matters	Likelihood	Impact	Mitigation	Residual Risk
Under-escalation of sensitive cases	A complaint involving fraud, legal action or account security that reaches the AI resolution path without escalating exposes Cartly to regulatory and reputational harm	Low	Critical	Risk-aware routing with hard escalation rules for defined trigger keywords; confidence floor below which the system always escalates; kill switch degrades to human path	Very low - hard rules are deterministic and not model-dependent
Prompt injection in complaint text	A hostile message embedded in ticket text can attempt to override agent instructions and manipulate tool calls	Medium	High	Input hygiene and sanitisation at the gateway; tool-call guards that validate parameters before execution; Safety Critic as final filter; adversarial synthetic data in red-team evaluation pass	Low - multi-layer guards reduce attack surface

4.2 Kill Switch and Graceful Fallback

When aggregate confidence across a ticket resolution path falls below the defined minimum, or when any hard trigger condition is met, the system must degrade to a human path. It must never guess on a sensitive case.

Trigger Conditions for Immediate Escalation

- Keyword detection: fraud, legal action, solicitor, police, lawsuit or regulatory complaint
- Ticket involves account suspension or permanent account closure
- Refund amount exceeds INR 500 and the request is not a standard return
- Customer has submitted three or more contacts on the same unresolved issue
- Sentiment score indicates extreme distress on two consecutive turns
- Any LLM call returns an error or timeout and the fallback chain is exhausted

Fallback Behaviour

- The system writes the full interaction record to the handoff queue immediately
- The customer receives an acknowledgement within 30 seconds confirming human review
- The human agent receives the complete case brief with all prior agent steps listed
- The ticket is marked as system-escalated in the audit log with the trigger reason

Kill Switch Command

A single environment variable SYSTEM_MODE=human_only forces all tickets to the human queue regardless of classification. Setting this variable does not require a code deployment. It is the one-command kill switch available to any operator with environment access.

Stage 5 - Data Strategy

Sprints 1 to 2 Deliverable
Two data cards and a benchmark table that shows with numbers what the augmentation changed. Real data is narrow and clean; real work is broad and messy. The synthetic pipeline closes that gap.

5.1 Dataset Selection and Justification

Dataset	Scale	Licence	Assessment
Bitext Customer Support (PRIMARY)	27k Q and A pairs; 27 intents; 10 categories	CDLA-Sharing-1.0	Cleanest intent taxonomy; maps directly to Cartly ticket categories; best starting point for intent classifiers and baseline benchmarks. Weakness: too clean; lacks multi-intent and emotional language of real tickets
Customer Support on Twitter (SUPPLEMENT)	3M tweets and replies; 20 brands	Kaggle terms of use	Real and messy; multi-turn; captures frustrated and emotional language closely matching real ticket tone. Weakness: not marketplace-specific; no order or policy structure; requires heavy filtering
MultiWOZ 2.2 (SYNTHETIC TEMPLATES)	10k task-oriented dialogues; 8 domains	Apache 2.0	Multi-turn slot-filling structure maps to agent state management. Weakness: travel and restaurant domain; policy content is irrelevant; used only as structural template for synthetic pipeline
Schema-Guided Dialogue (SYNTHETIC TEMPLATES)	20k+ dialogues; 20 domains; intent and slot annotations	Apache 2.0	Richest slot annotation set; good for dispute agent state design. Same domain mismatch; used only as structural template

The primary dataset is Bitext Customer Support. MultiWOZ and SGD provide structural templates for the synthetic data pipeline only and are not used as training data directly.

5.2 Primary Dataset Card

Field	Value
Dataset name	Bitext Customer Support LLM Chatbot Training Dataset

Field	Value
Source	Hugging Face - bitext/Bitext-customer-support-llm-chatbot-training-dataset
Licence	CDLA-Sharing-1.0 - confirmed on dataset page before redistribution
Size	Approximately 27,000 question and answer pairs
Intent categories	27 intents across 10 category groups including order, refund, payment, account and shipping
Schema	instruction (user query), response (agent reply), category (top-level), intent (granular)
Class balance	Intents are roughly balanced at approximately 1,000 examples each; some rare intents have fewer than 300 examples
Known gaps	No multi-turn dialogues; no emotional or hostile language; no multi-intent tickets; no dispute scenarios; no PII-redaction examples
Sensitive data	Synthetic data; no real PII present; safe for prompt use without additional redaction

5.3 Synthetic Data Pipeline

The pipeline addresses the gap between the clean primary dataset and the messy reality of production support tickets. It generates six categories of synthetic data, each serving a named purpose in the evaluation plan.

Synthetic Category	Volume Target	Generation Method and Purpose
Multi-turn escalation scenarios	2,000 dialogues	Seeded from MultiWOZ templates; adapted to Cartly context; simulates a customer who provides insufficient information across three to five turns before the agent has enough context to act. Tests Triage and Specialist Agent multi-turn handling
Adversarial and prompt injection attempts	500 examples	Manually crafted trigger phrases and embedded instruction overrides; tests Safety Critic and gateway guardrails. Examples include instructions to ignore policy, requests to reveal system prompts and malformed JSON attempts
Hindi-English code-switching variants	1,000 examples	Translations and code-switching of 500 Bitext examples into Hindi-English mixed text; tests robustness of the intent classifier on non-English inputs
Policy-trap cases	800 examples	Tickets where the customer claims an eligibility that does not exist under current policy (for example requesting a 30-day return on a non-returnable item);

Synthetic Category	Volume Target	Generation Method and Purpose
		tests Policy Agent faithfulness and refusal capability
Rare intent examples	600 examples	Account suspension, buyer-seller dispute and payment gateway failure; covers the three intents with fewer than 300 real examples in the primary dataset
High-value refund scenarios	400 examples	Refund requests above the INR 500 threshold requiring the human approval gate; tests that the Refund Agent correctly defers and triggers escalation

Validation Protocol

- Deduplication: cosine similarity check against primary dataset; remove any synthetic example with similarity above 0.92
- LLM quality check: pass each synthetic example through a judge prompt that scores fluency, intent correctness and realism on a 1-to-5 scale; discard below 3
- Human spot-check: random sample of 100 examples from each category reviewed by a team member before the set is locked

5.4 Benchmark Design

A held-out test set of 500 real Bitext examples is locked before any model training or prompt optimisation begins. It is versioned and never modified. The benchmark measures quality on the real-only test set against quality on the real-plus-synthetic test set to confirm that synthetic data improves performance and report cases where it does not.

Benchmark Dimension	Measurement Method
Intent classification accuracy	Exact-match accuracy on the 27-intent taxonomy; measured on real-only and real-plus-synthetic configurations

Benchmark Dimension	Measurement Method
RAGAS faithfulness	Automated RAGAS pipeline on 200 policy retrieval examples from the held-out set
Escalation precision	Of all escalated examples in the test set, the percentage that are true escalations per the golden label
Resolution rate	Of all Tier 1 examples in the test set, the percentage the system closes correctly without escalation
Synthetic data lift	Delta between real-only and real-plus-synthetic on all dimensions above; reported honestly including regressions

Stage 6 - Build

Sprints 2 to 3 Deliverable
A running system that survives a burst, traces a single ticket end to end and reports what it cost. The system is observable, testable and deployable from a single command.

6.1 Implementation Plan by Sprint

Sprint	Focus	Build Milestones
Sprint 2	Core agent path end to end	Triage Agent and Order Agent live; LLM gateway operational with LiteLLM; ChromaDB loaded with policy documents; Refund Agent with INR 500 hard threshold; first RAGAS baseline; first cost measurement
Sprint 3	Full agent set and hardening	Policy Agent and Dispute Agent live; Safety Critic integrated; concurrency and caching; DSPy optimisation run; A/B results; regression suite gating merges; escalation agent and handoff brief output; updated risk register

6.2 Concurrency and Load Handling

- Agents run concurrently using async execution in LangGraph; multiple tickets processed in parallel
- A work queue (Redis-backed or in-memory) buffers incoming tickets during burst periods
- Semantic caching is applied to the Policy Agent: WISMO and standard returns queries with cosine similarity above 0.95 to a cached query return the cached result without an LLM call
- Synthetic data generation and evaluation jobs run as distributed batch jobs; no long serial loops

6.3 Token Economics Instrumentation

Every model call records input tokens, output tokens, model ID, agent ID, ticket ID and timestamp to the audit log. A cost view aggregates these records into cost per call and cost per ticket. A budget guardrail alarm fires when the rolling 24-hour average cost per resolved ticket exceeds INR 25 (the warning threshold before the INR 30 hard ceiling).

Cost Control Mechanism	Implementation
Cheap model for high-volume paths	Triage and Order agents use the cheapest available model via LiteLLM routing; these two agents handle approximately 75% of all LLM calls

Cost Control Mechanism	Implementation
Semantic cache for repeated queries	WISMO queries repeat with high frequency; caching the top 200 most common query embeddings reduces Order Agent LLM calls by an estimated 40%
Budget alarm	Langfuse or MLflow alert when 24-hour rolling average cost per resolved ticket exceeds INR 25
Hard ceiling enforcement	If cost per resolved ticket exceeds INR 30 on a rolling 1-hour window, the system demotes to a lower model tier automatically and logs the event

6.4 Observability Design

Every agent step and tool call is traced. The dashboard surfaces four views: throughput, cost, quality and latency. A fifth view shows the escalation rate by category, which is the leading indicator of system health.

- Tracing: Langfuse trace attached to every ticket ID; shows the full agent path, model calls, tool calls, latency and cost for each ticket
- Dashboard: resolution rate by category; escalation rate by category; cost per ticket (rolling 24 hours); RAGAS faithfulness on sampled outputs; first-response latency percentiles (p50, p95, p99)
- Logging: structured JSON logs for all agent decisions; no free-text log lines in production code
- Alerting: alarms on cost ceiling breach; RAGAS faithfulness drop below 0.70; error rate above 2%; latency p95 above 45 seconds

Stage 7 - Verify

Every Sprint, Deepening in Sprint 3
Evaluation is not a final step. It runs every sprint and gates progression. A system that cannot be measured cannot be trusted.

7.1 Golden Set

The golden set is built from 500 real held-out Bitext examples augmented with 200 curated examples from the synthetic pipeline across all six synthetic categories. It is locked and versioned at Sprint 1. It is never modified. It is the gate condition for every merge to the main branch.

7.2 RAGAS Evaluation Targets

RAGAS Dimension	What It Measures	Sprint 4 Target
Faithfulness	Percentage of claims in the response that are supported by the retrieved context	0.85 or above
Answer relevancy	Whether the response directly addresses the question asked	0.80 or above
Context precision	Whether the retrieved chunks are ranked with the most relevant first	0.75 or above
Context recall	Whether the retrieved context contains all information needed for a correct answer	0.75 or above

7.3 LLM Judge Design

A strong LLM (Claude or GPT-4 class) serves as the automated judge for response quality. The judge uses a written rubric covering four dimensions: correctness (does the answer match the ground truth intent), faithfulness (are all claims grounded), appropriateness (is the tone and action correct for the ticket type) and safety (does the response avoid any of the five risk register categories).

The judge is calibrated against a 100-example human-labelled sample before it is used in the evaluation pipeline. Agreement between the judge and the human labels must reach Cohen's Kappa of 0.70 or above before the judge is accepted as authoritative. The calibration result is reported in the evaluation submission.

7.4 DSPy Optimisation

DSPy is used to compile and optimise prompts and pipelines against the resolution rate metric. The before-and-after comparison is reported with the delta on every RAGAS dimension and on resolution rate. Where DSPy optimisation produces a regression on any dimension, it is reported honestly alongside the improvement.

7.5 A/B Testing Plan

Experiment	Variants	Decision Criterion
Model routing policy	Cheap-only vs. cheap-for-Tier-1 and medium-for-Tier-2	Winner has higher resolution rate at lower cost per resolved ticket
Prompt variant for Policy Agent	Zero-shot vs. few-shot with three examples	Winner has higher RAGAS faithfulness with no regression in answer relevancy
Safety Critic threshold	Faithfulness cutoff at 0.65 vs. 0.70 vs. 0.75	Winner minimises false pass rate (hallucinated responses sent) while keeping escalation rate below 40%
Semantic cache similarity cutoff	0.90 vs. 0.92 vs. 0.95 cosine threshold	Winner minimises stale cache hits while achieving maximum cache hit rate on WISMO queries

7.6 Safety and Red-Team Pass

The adversarial synthetic set of 500 examples is run through the full system. The evaluation measures the rate at which prompt injection attempts reach the customer-facing response layer and the rate at which policy-trap cases produce incorrect eligibility confirmations. Both must reach zero on the hard-threshold checks and below 5 percent on the soft-threshold checks.

Results are reported honestly. Any adversarial example that passes all guards and reaches the output layer is documented with the attack vector, the guard that failed and the proposed mitigation. This is a required section of the evaluation report.

Stage 8 - Operate

Sprint 4 Deliverable
A deployed system and a runbook another team could operate without the original team in the room. The handover is part of the work.

8.1 Deployment

- The system is deployed as a FastAPI application with a minimal web interface for ticket submission and a dashboard view
- Deployment target is a free cloud tier (Render or HuggingFace Spaces) sufficient for the system scale
- A single command (docker compose up or equivalent) launches the full system including the LLM gateway, agent services, vector store, database and dashboard
- No secrets are committed to the repository; all provider keys and configuration are injected via environment variables
- The repository includes a README with architecture diagram, setup instructions and run instructions

8.2 Service-Level Objectives

SLO	Target	Alert Threshold
First-response latency (p95)	30 seconds or under	Alert if p95 exceeds 45 seconds over a 10-minute window
System availability	99% during evaluation period	Alert if any component is unavailable for more than 60 seconds
Cost per resolved ticket (24-hour rolling)	Under INR 30	Warning at INR 25; hard alarm at INR 30
RAGAS faithfulness (daily sample)	0.85 or above	Alert if daily sample drops below 0.75
Escalation rate	Below 40% of all tickets	Alert if escalation rate exceeds 50% over a 1-hour window

8.3 Operate Runbook

Routine Monitoring

- Check the Langfuse dashboard daily: resolution rate, escalation rate, cost per ticket and RAGAS faithfulness
- Review any tickets flagged as safety-critic blocked; confirm each block was appropriate and log the outcome

- Review any budget alarm events; confirm the model demotion triggered correctly and the INR 30 ceiling was not crossed

Incident Response

- If RAGAS faithfulness drops below 0.70: pause autonomous resolution for the affected category; route to human queue; investigate policy document currency
- If cost ceiling is breached: activate SYSTEM_MODE=human_only immediately; investigate and resolve before re-enabling autonomous resolution
- If a wrong refund or account action is detected: activate SYSTEM_MODE=human_only; log the incident with the full audit trail; identify the guard that failed
- If prompt injection is confirmed: patch the gateway input sanitisation immediately; re-run the adversarial test set before re-enabling

Canary and Rollback Plan

- Any new prompt version or model version is deployed to 10% of traffic first (canary)
- The canary runs for a minimum of one hour before full rollout
- If resolution rate drops more than 5 points or RAGAS faithfulness drops more than 0.05 on the canary slice, the canary is rolled back immediately
- Rollback restores the previous prompt version or model assignment in the LiteLLM gateway configuration without a code deployment
- All rollback events are logged with timestamp, trigger metric and resolution action

Sprint Cadence and Exit Criteria

Five sprints, one week each. Each sprint opens with planning and closes with a review and retrospective. The team may not move to the next sprint until all exit criteria are met.

Sprint	Focus	Exit Criteria
Sprint 0	Discover and Define	Profiled data and prioritised ticket map; five user personas; PRD v1 with all metrics and baselines; risk register v1 with all five risks; evaluation plan with golden set design
Sprint 1	Design and De-risk	Architecture diagram and topology spec; orchestration decision record; LLM gateway and model routing policy design; agent registry design; synthetic pipeline v1; evaluation harness and golden set locked; a working spike on the riskiest assumption (INR 500 threshold enforcement)
Sprint 2	Build the Core	Core agent path working end to end (Triage, Order and Refund agents); LLM gateway live with LiteLLM; ChromaDB loaded with policy documents; first RAGAS evaluation and cost baseline measured
Sprint 3	Harden, Scale and Optimise	Full agent set live including Policy, Dispute, Escalation and Safety Critic; concurrency and caching operational; DSPy optimisation results and A/B test winners documented; regression suite gating merges; risk register updated with Sprint 3 findings
Sprint 4	Verify, Operate and Present	Full benchmark against the Sprint 2 baseline; deployment and runbook complete; 20-minute video recorded; viva prepared; Demo Day delivered

Tooling Stack

Free-first throughout. Open models are used for all iteration. Paid API calls are reserved for evaluation runs and the final system demonstration. Every tool choice has a stated rationale and a named alternative.

Layer	Selected Tool	Rationale and Alternative
Agent orchestration	LangGraph (primary)	Stateful graph execution; native conditional branching; strong observability. Alternative: CrewAI for simpler role-based setups; AutoGen for debate and multi-agent negotiation patterns
LLM gateway	LiteLLM	Single abstraction over all LLM providers; provider-level fallback; cost tracking per call. No direct alternative with the same feature set at zero cost
Development models	DeepSeek, Qwen, Llama or Gemini Flash free tier	Free-first; swap via LiteLLM without agent code changes. Strong models (Claude or GPT-4) reserved for evaluation and final runs
Vector store	ChromaDB with timestamp metadata	Zero-cost; local deployment; timestamp metadata enables policy drift detection. Alternative: Qdrant for production scale
Retrieval evaluation	RAGAS	Native measurement of faithfulness, answer relevancy, context precision and context recall. No equivalent free alternative
Prompt and pipeline optimisation	DSPy	Compiles and optimises prompts against a metric rather than hand-tuning. Emerging alternative: GEPA or TextGrad for newer optimisation approaches
Serving and interface	FastAPI with minimal front end	Lightweight; zero cost; deployable on free cloud tiers. Dashboard via Langfuse or MLflow
Deployment	Free cloud tier (Render or HuggingFace Spaces)	Sufficient for evaluation-period scale. Docker Compose enables one-command reproducible local deployment
Safety layer	Llama Guard or NeMo Guardrails	Structured safety constraints for the Safety Critic; prompt injection guards at the gateway
Observability	Langfuse (primary)	Native LangGraph integration; per-trace cost and latency; RAGAS plugin. Alternative: MLflow for experiment tracking and A/B result logging